# Automation, ML or AI?

How to look at a problem and place it, with reasons, before anyone picks a tool.

60 min | Recorded | Class 2 of 15

# What this session covers

01
**Start from the problem**
4 slides

02
**Setting up a problem**
4 slides

03
**How machines learn**
6 slides

04
**Code walkthrough**
4 slides


© 2026 Devansh Parapalli

Section 1 of 4

# Start from the problem

The tool comes last.

01

Most AI projects that fail picked the tool before they understood the problem

02

The card has seven places a problem can land

03

Six questions place almost any problem on the card


© 2026 Devansh Parapalli

# Most AI projects that fail picked the tool before they understood the problem

“We should use AI for the service desk” is a tool looking for a job. It skips the questions that decide whether anything needs building at all.

Start with the problem written as one sentence. The tool falls out of the answers.

| | |
|---|---|
| **places on the card** | 7 |
| **questions to place a problem** | 6 |
| **sentence to start** | 1 |


© 2026 Devansh Parapalli

# The **cheapest option** that stays inside the error budget wins.

Cheapest to build and to run, counting the people who check it.


© 2026 Devansh Parapalli

# The card has seven places a problem can land

| | |
|---|---|
| **1. A script** | The steps are known and don’t change. |
| **2. A rule** | The decision is known and must be explainable line by line. |
| **3. Classical ML** | Labelled history, and the input is a table or short text. |
| **4. A small trained model** | Lots of text, images or audio, enough labels, cost per call matters. |
| **5. An LLM with a prompt** | Varied language in and out, few labels, someone checks the output. |
| **6. An LLM plus retrieval, tools or agents** | Needs private knowledge, has to act, or takes several steps. |
| **7. Not yet** | No ground truth, needs a guarantee, or nobody owns the result. |


© 2026 Devansh Parapalli

# Six questions place almost any problem on the card

1. Is the rule already written down?
2. What goes in: a table, text, or media?
3. Is there labelled history?
4. What does one wrong answer cost, and who notices?
5. Does the answer have to be exact, auditable or repeatable?
6. How often does it run, and what can we spend per run?


© 2026 Devansh Parapalli

Section 2 of 4

# Setting up a problem

Write it down so it can be judged.

01

Every problem gets set up the same way, before anyone argues about tools

02

Worked example: “has the import finished?”

03

Worked example: a payroll mismatch

04

Worked example: predicting reconciliation breaks


© 2026 Devansh Parapalli

# Every problem gets set up the same way, before anyone argues about tools

1

Write it as one sentence with a verb

2

Name what goes in and what should come out

3

Answer the six questions, honestly

4

Put a number on one wrong answer

5

Place it; write one line on why


© 2026 Devansh Parapalli

# Worked example: “has the import finished?”

Accounts payable asks the desk to check an InvoiceHub batch, several times a day.

**01 The answer is a lookup**
InvoiceHub already knows each batch’s status. Nobody needs to judge anything.

**02 Input is a batch ID**
A fixed shape. A regex finds it in the ticket text.

**03 Lands on 1**
A script that reads the status and replies. No model anywhere.


© 2026 Devansh Parapalli

# Worked example: a payroll mismatch

Summarise a 40-message thread so the RCA can start.

**01 No rule exists**
Every mismatch is different; what matters changes each time.

**02 Language in, language out**
Few labels, varied input, and the analyst reads it anyway.

**03 Lands on 5**
An LLM with a prompt. A person checks it before it goes in the RCA.


© 2026 Devansh Parapalli

# Worked example: predicting reconciliation breaks

Say which GL account will fail at the next close.

**01 Little ground truth**
Breaks are rare and each has its own cause. There’s almost nothing to learn from.

**02 Guarantee expected**
Controllers want a yes or no they can act on. A model can only give odds.

**03 Lands on 7**
Not yet. A pre-close checklist on the riskiest accounts does more today.


© 2026 Devansh Parapalli

Section 3 of 4

# How machines learn

Four paradigms, and the kinds of task they handle.

01

Four ways a machine learns, each answering one question

02

Most problems at LOL are one of eight task types

03

A lot of desk work needs a script and nothing else

04

Adding AI doesn’t fix missing data, missing owners or a need for guarantees


© 2026 Devansh Parapalli

# Four ways a machine learns, each answering one question

| | |
|---|---|
| **Supervised** | “Given examples with the right answer, predict the answer for a new one.” |
| **Unsupervised** | “Nobody labelled this. What groups or outliers are in it?” |
| **Self-supervised** | “Hide part of the data and predict it.” Language models learn this way. |
| **Reinforcement** | “Try, get a reward or a penalty, try again.” We only need to recognise it. |


© 2026 Devansh Parapalli

# Most problems at LOL are one of eight task types

- **Classify:** which queue, which priority
- **Predict a number:** hours to resolve
- **Group:** tickets that look alike
- **Spot the odd one:** a host behaving strangely
- **Forecast:** volume around period close
- **Rank:** which runbook fits best
- **Pull out fields:** PO, batch, worker ID
- **Write:** a summary, a reply, a draft


© 2026 Devansh Parapalli

# A lot of desk work needs a script and nothing else

**Status checks.** InvoiceHub batch status, export job results. Read it, report it.

**Runbook steps.** Rerun the export, extend the CW Portal account. Same steps every time.

**Routing by a field.** The Gatehouse form already says “approver rights”. Read the field.


© 2026 Devansh Parapalli

# Adding AI doesn’t fix missing data, missing owners or a need for guarantees

- **No ground truth:** nothing to check its answers against
- **No owner:** nobody signs off on what it decides
- **Needs a guarantee:** a model gives likelihoods
- **Cause and effect:** history shows what went together, not what caused what


© 2026 Devansh Parapalli

# Place twelve LOL problems on the card

In pairs, take the twelve problems on the next slide. For each one, pick the cheapest place on the card that stays inside its error budget, and write one sentence on why.

10
minutes

**Expected output**

Twelve placements, each with one line of reasoning. “No AI needed” is a valid answer and often the right one.


© 2026 Devansh Parapalli

# Twelve problems to place

1. Restart the InvoiceHub import job after three failed health checks in a row
2. Route Gatehouse requests where the form already names the access type
3. Alert when the Ledgerline close job runs 30 minutes past its usual finish
4. Forecast ticket volume for the five days around period close
5. Spot odd CPU and latency across our 60 integration hosts
6. Route free-text service desk tickets using two years of history
7. Read fields from 50,000 labelled scanned A.S.S. invoices
8. Summarise a 40-message payroll mismatch thread for the RCA
9. Draft the first reply to a contractor waiting on Harbor MDM enrolment
10. Answer “how do I reopen a PO” from 600 Procura runbooks
11. Check the S.C. billing feed and open a posting ticket from chat
12. Say which GL account will fail reconciliation at the next close


© 2026 Devansh Parapalli

Section 4 of 4

# Code walkthrough

The six questions as a small script.

01

The six questions fit in a function that returns a place on the card

02

The script is a checklist that talks back, not an oracle

03

Today covered the whole card

04

What to do next


© 2026 Devansh Parapalli

# The six questions fit in a function that returns a place on the card

code/class02/place.py

```python
def place(p: Problem) -> int:
    if p.needs_guarantee or not p.has_ground_truth: return 7
    if p.rule_known: return 1 if p.steps_only else 2
    if p.input == "table" and p.has_labels: return 3
    if p.input == "text" and p.has_labels and p.high_volume: return 3
    if p.input == "media" and p.has_labels and p.high_volume: return 4
    if p.needs_private_knowledge or p.must_act: return 6
    return 5
```


© 2026 Devansh Parapalli

# The script is a checklist that talks back, not an oracle

It encodes the order we ask the questions in: rule out “not yet” first, then the cheapest options, and only then the LLM. Real problems will argue with it. That argument is the point of the exercise.


© 2026 Devansh Parapalli

# Today covered the whole card

| | |
|---|---|
| **1. A script** | **The steps are known and don’t change.** |
| **2. A rule** | **The decision is known and must be explainable line by line.** |
| **3. Classical ML** | **Labelled history, and the input is a table or short text.** |
| **4. A small trained model** | **Lots of text, images or audio, enough labels, cost per call matters.** |
| **5. An LLM with a prompt** | **Varied language in and out, few labels, someone checks the output.** |
| **6. An LLM plus retrieval, tools or agents** | **Needs private knowledge, has to act, or takes several steps.** |
| **7. Not yet** | **No ground truth, needs a guarantee, or nobody owns the result.** |


© 2026 Devansh Parapalli

# What to do next

1. Place the problem you brought on the card, and write the one line on why.
2. Read the class 3 pre-reading on train and test splits.
3. Run `code/class02/place.py` and add your own problem to it.

Questions after class: the course channel.


© 2026 Devansh Parapalli